PROJECT

on

E-Campus

Submitted By

RAHUL VEGDANI – 56

VISHWAJIT WAGHDHARE – 57

SOHAM TAMBADE – 52

PARIDHI SUREKA – 51

ASHUTOSH PANDEY – 36

Under the esteemed guidance of

Dr. Girish Tere

Submitted in partial fulfilment of the requirements

For MCA, Semester III Examination



Aditya Institute of Management Technology & Research,

Borivali (West), Mumbai – 400092.

University of Mumbai | Academic Year 2025–2027

ADITYA
INSTITUTE OF MANAGEMENT STUDIES AND RESEARCH
AIMSR
NAAC Accredited || ISO 21001 : 2018 Certified

Date: _____

CERTIFICATE

This is to certify that **Mr. Rahul Vegdani,** student of Master of Computer Application (MCA) of Aditya Institute of Management Technology & Research, **Roll No. 56**, has completed and submitted the E-Campus project in partial fulfilment of the requirements as per the syllabus defined by the University of Mumbai for the academic year 2025–2027.

It is further certified that the project work has been submitted for academic evaluation.

Examiner Professor Head of Department External In-charge

ADITYA
INSTITUTE OF MANAGEMENT STUDIES AND RESEARCH
AIMSR
NAAC Accredited || ISO 21001 : 2018 Certified

Date: _____

CERTIFICATE

This is to certify that **Mr. Vishwajit Waghdhare**, student of Master of Computer Application (MCA) of Aditya Institute of Management Technology & Research, **Roll No. 57**, has completed and submitted the E-Campus project in partial fulfilment of the requirements as per the syllabus defined by the University of Mumbai for the academic year 2025–2027.

It is further certified that the project work has been submitted for academic evaluation.

Examiner Professor Head of Department External In-charge

ADITYA
INSTITUTE OF MANAGEMENT STUDIES AND RESEARCH
AIMSR
NAAC Accredited || ISO 21001 : 2018 Certified

Date: _____

CERTIFICATE

This is to certify that **Mr. Soham Tambade**, student of Master of Computer Application (MCA) of Aditya Institute of Management Technology & Research, **Roll No. 52**, has completed and submitted the E-Campus project in partial fulfilment of the requirements as per the syllabus defined by the University of Mumbai for the academic year 2025–2027.

It is further certified that the project work has been submitted for academic evaluation.

Examiner Professor Head of Department External In-charge

ADITYA
INSTITUTE OF MANAGEMENT STUDIES AND RESEARCH
AIMSR
NAAC Accredited || ISO 21001 : 2018 Certified

Date: _____

CERTIFICATE

This is to certify that **Ms. Paridhi Sureka**, student of Master of Computer Application (MCA) of Aditya Institute of Management Technology & Research, **Roll No. 51**, has completed and submitted the E-Campus project in partial fulfilment of the requirements as per the syllabus defined by the University of Mumbai for the academic year 2025–2027.

It is further certified that the project work has been submitted for academic evaluation.

Examiner Professor Head of Department External In-charge

ADITYA
INSTITUTE OF MANAGEMENT STUDIES AND RESEARCH
AIMSR
NAAC Accredited || ISO 21001 : 2018 Certified

Date: _____

CERTIFICATE

This is to certify that **Mr. Ashutosh Pandey**, student of Master of Computer Application (MCA) of Aditya Institute of Management Technology & Research, **Roll No. 36**, has completed and submitted the E-Campus project in partial fulfilment of the requirements as per the syllabus defined by the University of Mumbai for the academic year 2025–2027.

It is further certified that the project work has been submitted for academic evaluation.

Examiner Professor Head of Department External In-charge

Contents

DECLARATION 10
STATEMENT BY THE CANDIDATES 11
ACKNOWLEDGEMENT 12
ABSTRACT 13
CHAPTER 1: INTRODUCTION 14
1.1 Background 14
1.2 About the Institution and Project Context 14
1.3 Problem Definition 14
1.4 Objectives 14
1.5 Scope 14
1.6 Existing System 15
1.7 Proposed System 15
1.8 Assumptions and Constraints 15
1.9 Feasibility Study 15
1.9.1 Technical Feasibility 15
1.9.2 Operational Feasibility 15
1.9.3 Economic Feasibility 15
1.10 Chapter Summary 16
CHAPTER 2: LITERATURE AND TECHNOLOGY REVIEW 17
2.1 Introduction 17
2.2 Mobile Campus Applications 17
2.3 Flutter and Dart 17
2.4 Firebase Authentication 17
2.5 Cloud Firestore 17
2.6 Riverpod 17
2.7 Security and Role-Based Access 17
2.8 Comparative Summary 17
2.9 References for Technology Review 18
2.10 Chapter Summary 18
CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION (SRS) 19
3.1 Purpose 19
3.2 System Overview 19
3.3 User Categories 19
3.4 Functional Requirements 19

3.5 Non-Functional Requirements 20
3.6 Hardware Requirements 20
3.7 Software Requirements 20
3.8 Constraints and Dependencies 21
3.9 Acceptance Criteria 21
3.10 Chapter Summary 21
CHAPTER 4: SYSTEM ANALYSIS AND DESIGN 22
4.1 Development Methodology 22
4.2 High-Level Architecture 22
4.3 Application Activity Flow 23
4.4 Use-Case Diagram 24
4.5 Data-Flow Diagram 24
4.6 Role-Based Access Matrix 24
4.7 Data Model and Firestore Design 25
4.8 Notice Entity — Conceptual Fields 25
4.9 Security Design 26
4.10 UI Design and Navigation 26
4.11 Chapter Summary 27
CHAPTER 5: MODULE DESCRIPTION AND IMPLEMENTATION 28
5.1 Application Initialisation 28
5.2 Authentication Module 28
5.3 User Profile and Role Resolution 29
5.4 Role-Based Home and Navigation 29
5.5 Notices Module 30
5.5.1 Notice Lifecycle 30
5.6 Timetable Module 31
5.7 Attendance Module 32
5.8 Library / E-Library Module 33
5.9 Campus Chat 34
5.10 Notifications and AI Assistant 35
5.11 Project Source Structure 36
5.12 Chapter Summary 37
CHAPTER 6: TESTING AND RESULTS 38
6.1 Testing Objectives 38
6.2 Testing Types 38

6.3 Test Cases 38
6.4 Test Environment Record 39
6.5 Results and Evidence 39
6.6 Chapter Summary 40
CHAPTER 7: INSTALLATION, CONFIGURATION AND EXECUTION 41
7.1 Prerequisites 41
7.2 Verify the Development Environment 41
7.3 Firebase Configuration 41
7.4 Install Dependencies 41
7.5 Run the Application 41
7.6 Build an Android APK 41
7.7 Common Troubleshooting 42
7.8 Chapter Summary 42
CHAPTER 8: LIMITATIONS AND FUTURE ENHANCEMENTS 43
8.1 Current Limitations 43
8.2 Future Enhancements 43
8.3 Conclusion 43
REFERENCES 44

DECLARATION

We, Rahul Vegdani, Vishwajit Waghdhare, Soham Tambade, Paridhi Sureka and Ashutosh Pandey, hereby declare that the project entitled “E-Campus”, submitted in partial fulfilment of the requirements for the award of Master of Computer Application during the academic year 2025–2027, is our original work and has not formed the basis for the award of any degree, associateship, fellowship or other similar title.

Student Signatures:

1. Mr. Rahul Vegdani (Roll No. 56): _____
2. Mr. Vishwajit Waghdhare (Roll No. 57): _____
3. Mr. Soham Tambade (Roll No. 52): _____
4. Ms. Paridhi Sureka (Roll No. 51): _____
5. Mr. Ashutosh Pandey (Roll No. 36): _____

Place: Mumbai

Date: _____

STATEMENT BY THE CANDIDATES

We wish to state that the work embodied in this report titled “E-Campus” forms our contribution to the project work conducted under the guidance of Dr. Girish Tere at Aditya Institute of Management Technology & Research. We declare that this submission represents our work and that ideas or words from other sources have been cited and referenced appropriately. We further declare that we have followed the principles of academic honesty and integrity and have not knowingly misrepresented, fabricated, or falsified any idea, data, fact, or source in this submission.

1. Mr. Rahul Vegdani (Roll No. 56): _____
2. Mr. Vishwajit Waghdhare (Roll No. 57): _____
3. Mr. Soham Tambade (Roll No. 52): _____
4. Ms. Paridhi Sureka (Roll No. 51): _____
5. Mr. Ashutosh Pandey (Roll No. 36): _____

Place: Mumbai

Date: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Dr. Girish Tere, for her guidance, advice, and support throughout the development of E-Campus. We also thank the faculty members, principal, and staff of the Department of MCA at Aditya Institute of Management Technology & Research for their assistance and encouragement. Finally, we thank our families and friends for their support throughout the project.

Rahul Vegdani

Vishwajit Waghdhare

Soham Tambade

Paridhi Sureka

Ashutosh Pandey

ABSTRACT

E-Campus is a Flutter-based college management mobile application designed to provide students, teachers, and administrators with a common digital interface for selected campus and academic workflows. The project uses Dart and Flutter for the client application, Firebase services for authentication and cloud data storage, and Riverpod for application state management. The project context includes role-based navigation and modules such as notices, timetable, attendance, library resources, and service components for AI assistance and notifications. The current report documents the intended system, available implementation context, design, requirements, setup, and verification plan.

The application aims to make campus information easier to access and to support role-appropriate management of academic content. The notices workflow has recently been tested by the project group: administrators can delete any notice, while teachers can delete their own notices. Other module capabilities are described with appropriate qualification where final implementation evidence has not been supplied. The report distinguishes confirmed behaviour from planned or unverified functionality.

This document is a detailed working report. Before final academic submission, the group should verify all module statuses against the exact build, insert genuine application screenshots, complete pending test results, and confirm institutional formatting requirements.

CHAPTER 1: INTRODUCTION

1.1 Background

Educational institutions coordinate announcements, academic schedules, attendance information, library resources, and communication among students, teachers, and administrators. When these activities are distributed across separate tools or manual channels, users may need to search in several places and staff may need to repeat updates. A mobile-first application can offer one access point for selected workflows while retaining access restrictions for different user roles.

1.2 About the Institution and Project Context

The supplied reference report identifies Aditya Institute of Management Technology & Research (AIMTR/AIMSR naming appears inconsistently in the reference) in Borivali (West), Mumbai, as the institutional context. This report uses the formal name shown on the reference cover page: Aditya Institute of Management Technology & Research. Confirm the exact institutional naming required by the college before submission.

E-Campus is the project title. The application is built with Flutter and Dart, with Firebase-related services and Riverpod in the known project context. It is intended to support role-specific campus workflows through a single mobile application.

1.3 Problem Definition

Campus information can be fragmented among notices, schedules, attendance records, library resources, and messaging channels. Students may have difficulty locating current information, while staff may need to maintain and distribute updates through separate processes. The project addresses the need for a unified application interface for the campus workflows included in its current scope.

1.4 Objectives

- Provide a mobile interface for selected campus-management workflows.
- Support role-aware navigation for student, teacher, and administrator accounts.
- Use Firebase-backed identity and data services where configured.
- Provide a notice workflow with role- and ownership-based management.
- Present selected academic information such as timetable, attendance, and library content where supported by the submitted build.
- Organise the code into reusable screens, models, repositories/services, and shared configuration.
- Establish a maintainable base for future campus features.

1.5 Scope

This report focuses on the Flutter/Firebase E-Campus application represented by the project context and the supplied reference report. It covers authentication, role-based navigation, notices, timetable, attendance, library screens, architecture, database considerations, setup, and testing. Additional features described in the reference such as bus tracking, advanced AI, placement, canteen, grievance, and extended analytics—must be treated as planned, partial, or unverified unless the final build demonstrates them.

1.6 Existing System

In a conventional process, announcements, schedules, attendance information, and learning resources may be shared through separate systems. This can make information retrieval less consistent and can increase the effort needed to publish or update information. This description is a general problem context; the group should adapt it if the institution requires a description of a specific existing system.

1.7 Proposed System

E-Campus proposes a common Flutter interface backed by cloud services. After authentication, the application resolves the user profile and role and presents the relevant screens. Data access is handled through the application service/repository layer and enforced by backend security rules. Staff notice management is subject to author ownership and administrator permissions.

1.8 Assumptions and Constraints

Type	Description
Assumptions	Users have a compatible device and basic smartphone skills; internet access is available for cloud-backed functions; Firebase is configured for the correct project.
Constraints	Feature availability depends on current code, backend configuration, published security rules, network access, and any external API configuration.
Security constraint	Hiding a UI control is not sufficient authorisation; backend rules must also reject unauthorised operations.
Academic constraint	Only features demonstrated and verified in the submitted build should be claimed as completed.

1.9 Feasibility Study

1.9.1 Technical Feasibility

Flutter supports mobile application development with Dart. Firebase Authentication and Cloud Firestore provide managed identity and document-data services. Riverpod can organise application state. These technologies are suitable for a prototype and can be expanded when implementation, security, and operational requirements are validated.

1.9.2 Operational Feasibility

A mobile interface can make supported campus information accessible through a familiar device. Role-specific navigation can reduce irrelevant controls, provided that the underlying access rules remain aligned with the user roles.

1.9.3 Economic Feasibility

The project uses commonly available development tools. Actual cloud and API costs depend on service usage, plan limits, and enabled integrations; no fixed cost estimate is asserted in this report.

1.10 Chapter Summary

This chapter introduced the project context, problem, objectives, scope, proposed approach, and feasibility considerations. The next chapter reviews related concepts and explains the requirements of the application.

CHAPTER 2: LITERATURE AND TECHNOLOGY REVIEW

2.1 Introduction

This chapter reviews the technology concepts relevant to E-Campus. The discussion is a technology-oriented review rather than a claim that a formal systematic literature review was completed. If the college requires peer-reviewed research papers, add verified papers and citations before submission.

2.2 Mobile Campus Applications

Mobile campus applications can consolidate announcements, academic schedules, learning resources, and student services in a single interface. Their usefulness depends on data accuracy, user experience, access control, and whether users can reliably access the service.

2.3 Flutter and Dart

Flutter is a UI toolkit used to build applications from a shared codebase, and Dart is its programming language. In E-Campus, Flutter widgets provide the presentation layer. A shared UI approach can support maintainability, although each target platform still requires appropriate configuration and testing.

2.4 Firebase Authentication

Firebase Authentication provides identity-management capabilities. The project context references an OTP-oriented login workflow in earlier work; the exact sign-in provider and production configuration should be checked in the final implementation. Authentication establishes identity but does not alone define which Firestore documents a user may access.

2.5 Cloud Firestore

Cloud Firestore is a document-oriented cloud database. Collections contain documents, and documents contain fields. Queries and security rules must be designed together: the rules determine which records a user can access, and a query must be compatible with those rules.

2.6 Riverpod

Riverpod is a state-management and dependency-management library for Flutter. It can help separate UI rendering from state and data dependencies. The current source should be inspected before describing the precise provider architecture.

2.7 Security and Role-Based Access

Role-based access control associates actions with user roles. In a client application, role-aware navigation helps present relevant screens, but backend rules must independently enforce read, create, update, and delete permissions. Ownership checks are important for records created by individual teachers.

2.8 Comparative Summary

Technology / concept	Role in E-Campus	Consideration
Flutter / Dart	Mobile presentation and application logic	Test the actual target platforms and device sizes.
Firebase Authentication	User identity and session	Confirm the configured

		provider and recovery/logout behaviour.
Cloud Firestore	Cloud document storage and queries	Document collection names and deployed rules accurately.
Riverpod	State and dependency management	Verify providers in current source code.
External APIs	Optional integrations such as maps or AI	Document only integrations enabled and tested in the final build.

2.9 References for Technology Review

Official documentation is listed in the References chapter. The group should add any required academic papers with full author, title, publication, year, and DOI/URL details only after verifying the original sources.

2.10 Chapter Summary

The chapter summarised the key technologies and security concepts relevant to E-Campus. These technologies provide a foundation, but successful implementation depends on correct configuration, access rules, and testing.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

3.1 Purpose

This SRS describes the expected behaviour and quality attributes of the E-Campus application. It is intended to guide implementation, testing, and academic evaluation. Requirements are phrased at system level and should be adjusted to match the final build.

3.2 System Overview

E-Campus is a Flutter client that communicates with Firebase services for identity and data access. The app uses role-oriented navigation for student, teacher, and administrator users. The final implementation may include additional integrations; their presence must be confirmed before being included as mandatory requirements.

3.3 User Categories

User category	**Primary responsibilities in the current report scope**
Student	Authenticate, view permitted notices, and access academic screens that are available to the account.
Teacher	Authenticate, view permitted campus information, and manage notices authored by the teacher.
Administrator	Authenticate, access administrative screens, and manage notices across authors.

3.4 Functional Requirements

ID	**Requirement**
FR-01	The application shall initialise its required services before presenting cloud-backed screens.
FR-02	The application shall provide the configured authentication flow.
FR-03	The application shall resolve the signed-in user profile and role.
FR-04	The application shall present navigation appropriate to the resolved role.
FR-05	The application shall display notices permitted for the current user.
FR-06	A teacher shall be able to delete a notice authored by that teacher, subject to backend rules.
FR-07	An administrator shall be able to delete any

	notice, subject to backend rules.
FR-08	A student shall not be offered the notice-delete control, and backend rules shall deny unauthorised deletion.
FR-09	The application shall display feedback for successful or failed data operations.
FR-10	Academic modules shall display only data that is available and authorised for the current account.
FR-11	The app shall provide a predictable way to navigate between available screens.
FR-12	Configuration and error handling shall avoid exposing private credentials.

3.5 Non-Functional Requirements

Category	Requirement
Security	Backend rules must validate authenticated identity, role, ownership, and allowed operations.
Usability	Controls should have understandable labels and confirmation for destructive actions.
Reliability	The UI should communicate failures and avoid false success messages.
Maintainability	Related logic should be organised into screens, models, services/repositories, and shared configuration where applicable.
Performance	Queries and screen updates should be responsive for expected usage.
Compatibility	The release must be built and tested using compatible Flutter, Dart, Android SDK, Gradle, and JDK versions.
Privacy	The report and screenshots must not disclose credentials or personal records unnecessarily.

3.6 Hardware Requirements

- Development computer capable of running Flutter and Android Studio.
- Android phone or emulator compatible with the project configuration.
- Internet connectivity for cloud-backed features.

3.7 Software Requirements

Component	Known project context (verify final checkout)

Framework / language	Flutter / Dart
Authentication	Firebase Authentication
Database	Cloud Firestore
State management	flutter_riverpod / Riverpod
Navigation	go_router is listed in prior project dependency context
Other dependencies	Firebase Storage, Firebase Messaging, Firebase App Check, intl, file_picker, url_launcher, http, flutter_markdown, shared_preferences — verify pubspec.yaml
Tools	Android Studio, Flutter SDK, Dart SDK, Git as applicable

3.8 Constraints and Dependencies

- Cloud features require valid Firebase configuration and internet access.
- Firestore rules must be published to the correct Firebase project.
- External integrations require valid configuration and may have usage limits or charges.
- The report must match the version of the code submitted for evaluation.

3.9 Acceptance Criteria

- The application starts with the configured Firebase project.
- The configured authentication flow can be demonstrated.
- Role-specific navigation matches the signed-in account.
- Notice deletion behaves as confirmed for teacher and administrator roles.
- Unauthorised deletion is denied by backend rules.
- Screenshots and test results correspond to the same build.

3.10 Chapter Summary

The SRS defines functional and non-functional requirements and identifies the key acceptance criteria for the current report scope.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

4.1 Development Methodology

An iterative, module-oriented development approach is appropriate for E-Campus because authentication, navigation, notices, and academic screens can be implemented and tested separately. The report describes this as an iterative approach; use the term Agile as the formal methodology only if the group actually followed and can explain an Agile process.

4.2 High-Level Architecture

The system is described using a client-and-cloud architecture. The Flutter interface accepts user actions and renders screen state. The repository/service layer coordinates data requests. Firebase Authentication handles identity, while Cloud Firestore stores application documents and applies security rules. External services are shown as optional because the exact enabled integrations need verification.

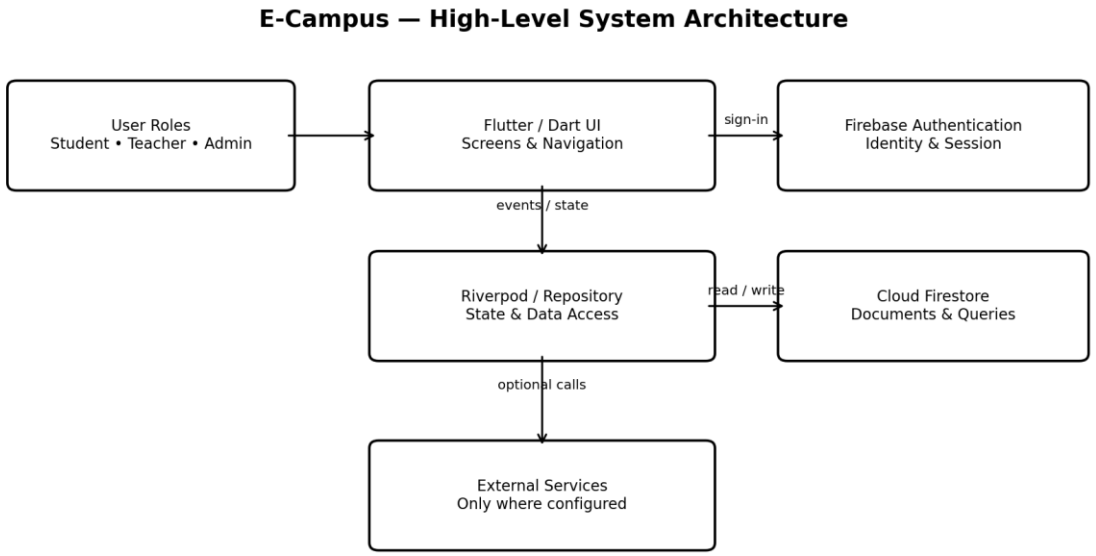


Figure 4.1: E-Campus high-level architecture (conceptual; validate against final source).

4.3 Application Activity Flow

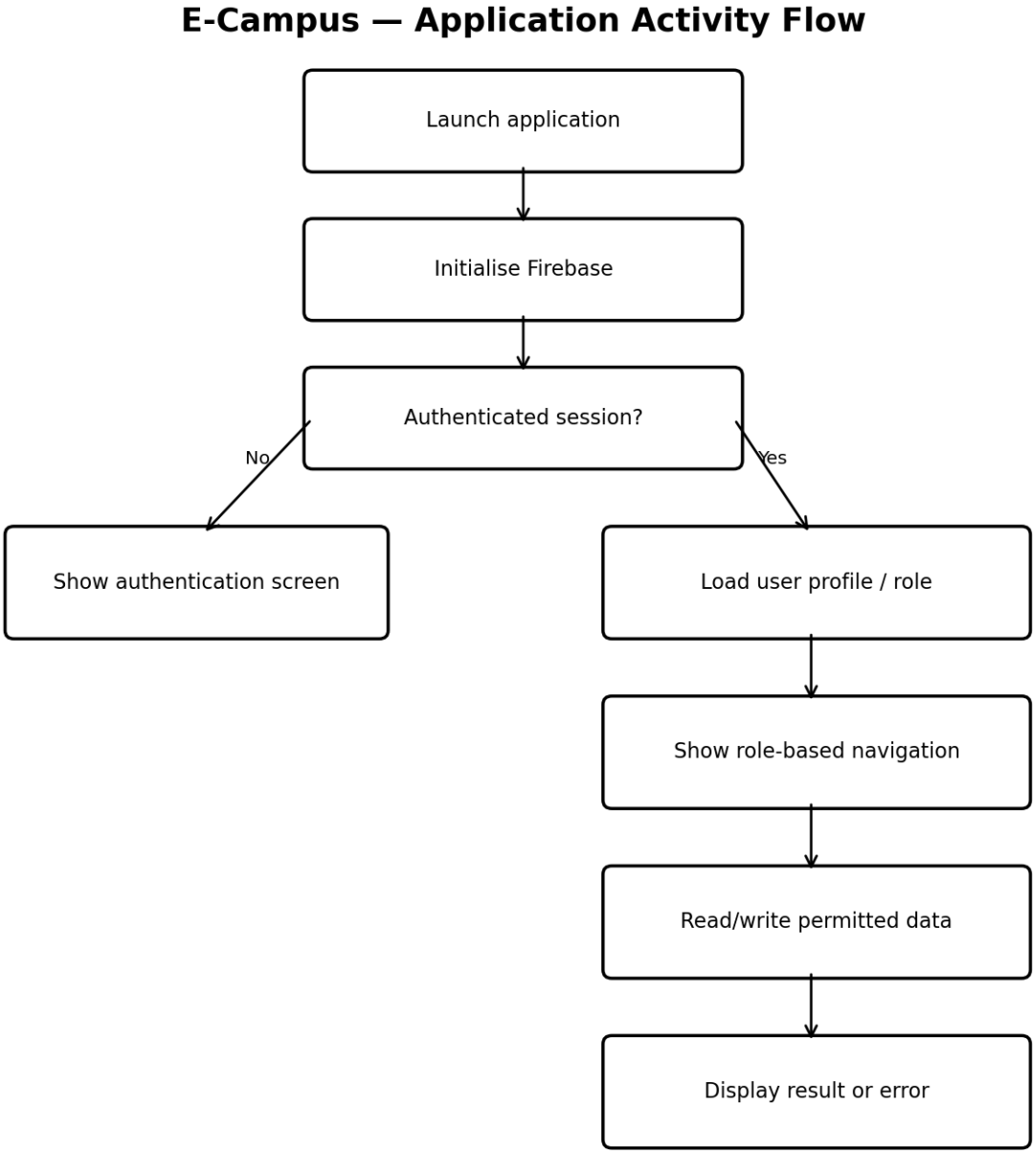


Figure 4.2: Application startup and role-based activity flow.

4.4 Use-Case Diagram

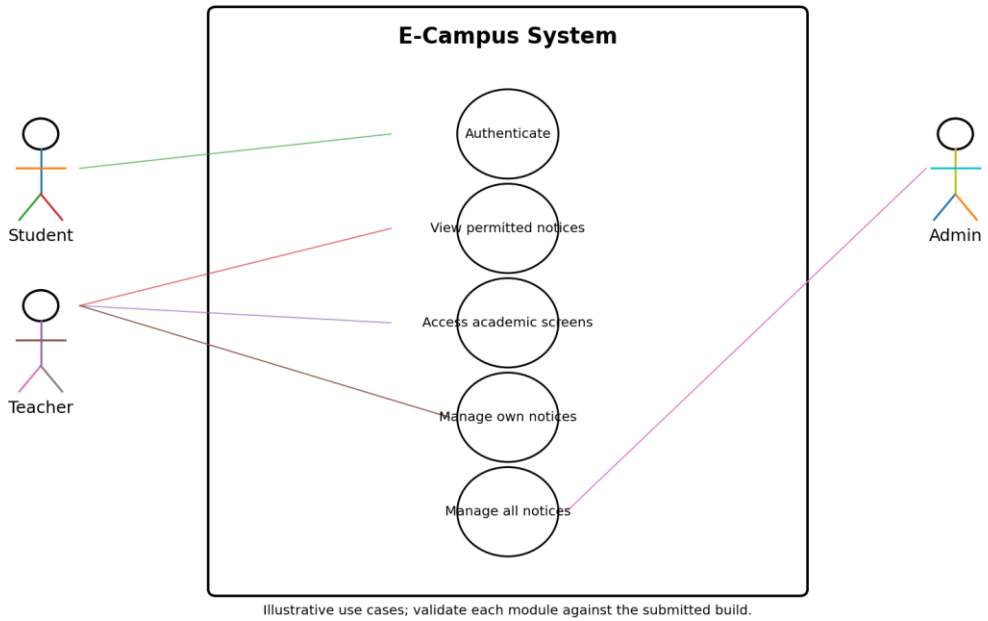


Figure 4.3: Illustrative use-case diagram for the current report scope.

4.5 Data-Flow Diagram

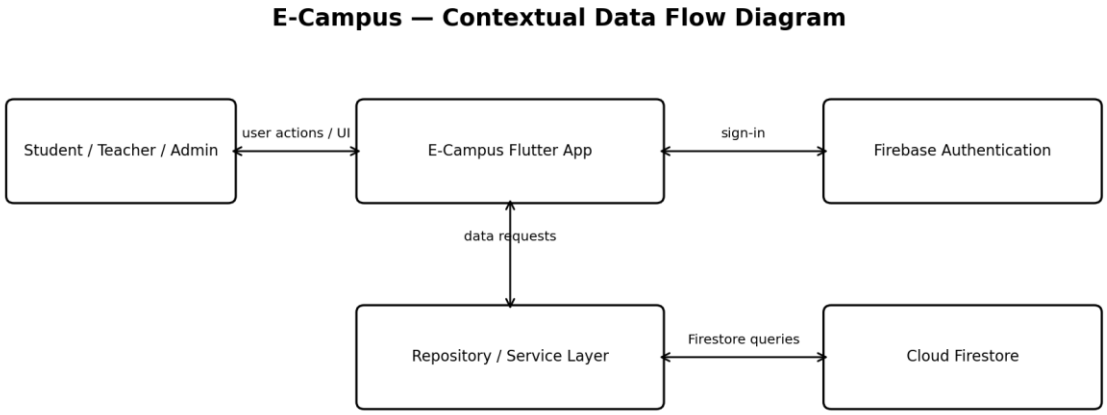


Figure 4.4: Contextual data-flow diagram for identity and Firestore operations.

4.6 Role-Based Access Matrix

Operation	Student	Teacher	Administrator
View permitted published notices	Yes, subject to rules	Yes, subject to rules	Yes, subject to rules

Create notice	No staff privilege expected	If enabled in current build	If enabled in current build
Edit own draft	No	If enabled in current build	Yes, subject to implementation
Delete own notice	No	Confirmed working	Yes
Delete another teacher's notice	No	No	Confirmed intended and tested
Access academic screens	According to account/data	According to account/data	According to account/data

4.7 Data Model and Firestore Design

Firestore is schemaless at the database level, so the application's model classes, repository queries, and security rules collectively define the practical data schema. The exact current collection inventory should be extracted from the latest repository and Firestore rules before submission. The following is a provisional design overview based on available project context.

Collection / area	Purpose	Verification needed
users / profile data	User profile and role resolution	Confirm exact collection name, UID key, and fields.
notices	Campus notices, author, publication state, and timestamps	Confirm required fields and indexes in current code.
timetable-related data	Timetable groups, requirements, and timetable entries are referenced in prior project context	Confirm exact collection names and document fields.
attendance-related data	Attendance screens and logic have been referenced	Confirm exact collections and calculations.
library-related data	Library/E-Library screen has been referenced	Confirm data source and file-storage workflow.
chat-related data	Campus Chat was part of scope and earlier permission debugging	Confirm current implementation status and collections.

4.8 Notice Entity — Conceptual Fields

Field	Description	Status
noticeId	Unique Firestore document identifier	Document ID, if used
title	Notice heading	Confirm exact field
body / content	Notice text	Confirm exact field
authorUid	UID of notice creator; used in	Previously referenced; verify

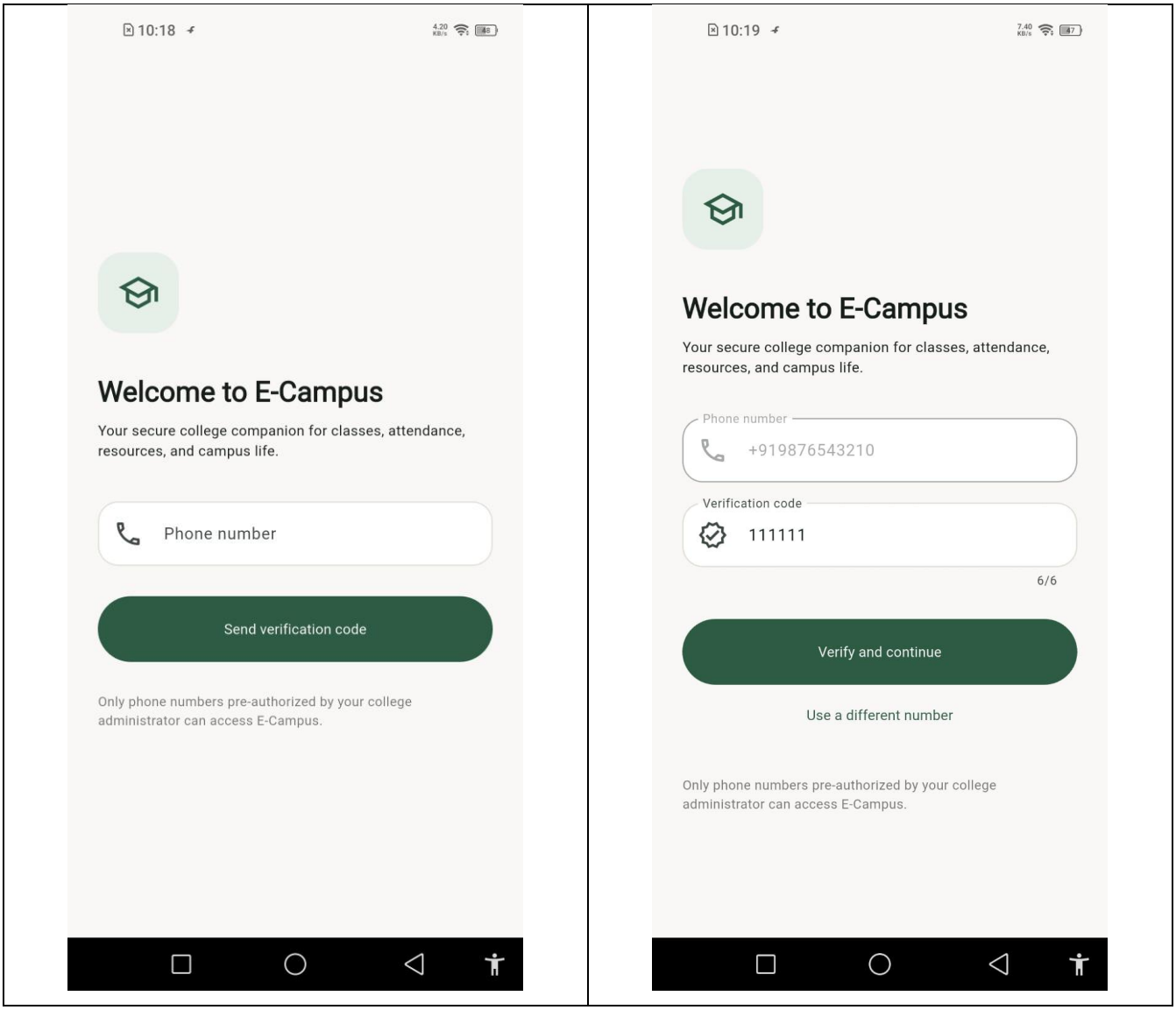
	prior rules context	current schema
published	Indicates published or draft state	Previously referenced; verify current schema
createdAt / updatedAt	Timestamps for lifecycle tracking	Confirm exact fields

4.9 Security Design

- Authenticate users before allowing protected data operations.
- Use Firestore rules to enforce ownership and role checks on the backend.
- Treat client-side visibility conditions as usability controls, not the only security boundary.
- Test both allowed and denied operations with student, teacher, and administrator accounts.
- Publish rules to the Firebase project actually used by the app.
- Avoid exposing service credentials and API secrets in source code or screenshots.

4.10 UI Design and Navigation

The user interface is organised into screens and role-based destinations. The report should include screenshots from the actual app for login, each role’s home screen, notices, timetable, attendance, and library screens that are available in the submitted build.



4.11 Chapter Summary

This chapter presents the conceptual architecture, activity flow, use cases, data flow, role matrix, provisional data model, and security approach. Diagrams are explanatory and should be reconciled with the exact final code before submission.

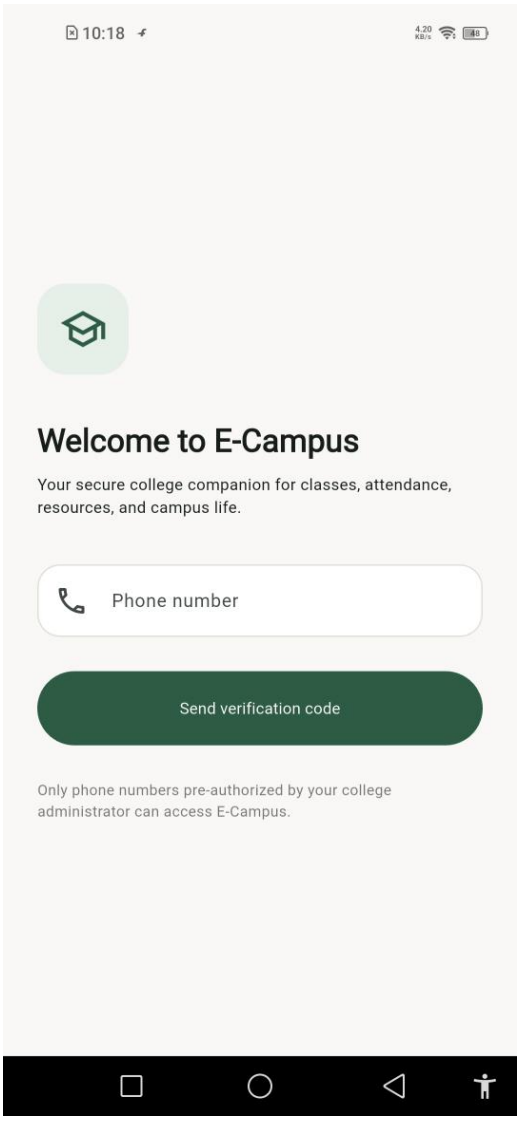
CHAPTER 5: MODULE DESCRIPTION AND IMPLEMENTATION

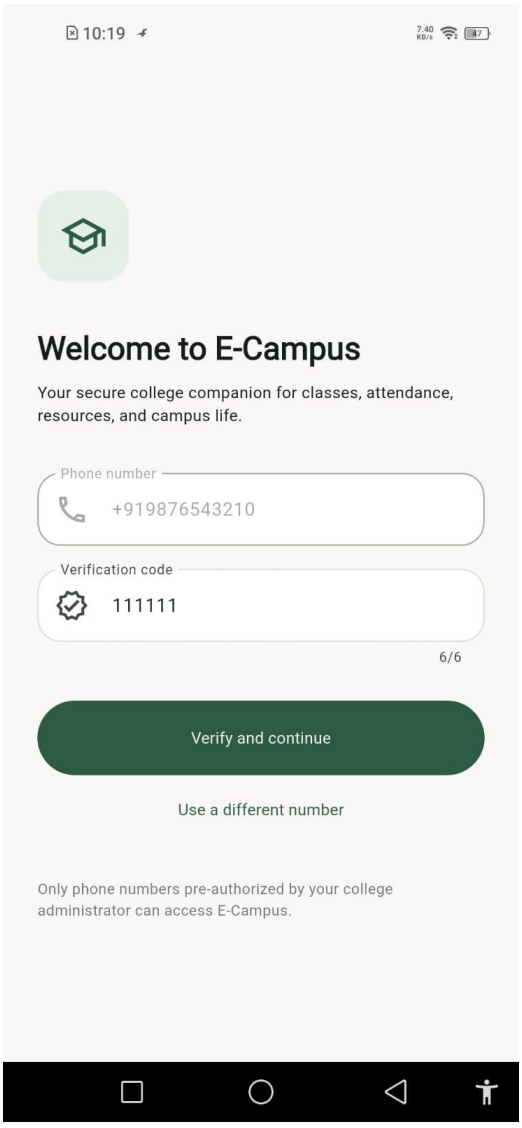
5.1 Application Initialisation

The Flutter application starts from its entry point and initialises required services before showing the authenticated experience. Firebase configuration is generated or supplied for the supported platform. The final implementation should be checked for the exact startup order and error handling.

5.2 Authentication Module

Authentication establishes the user identity used by subsequent profile resolution and data access. Earlier project context refers to OTP login; the group should confirm the exact provider, verification process, and account lifecycle in the submitted build.



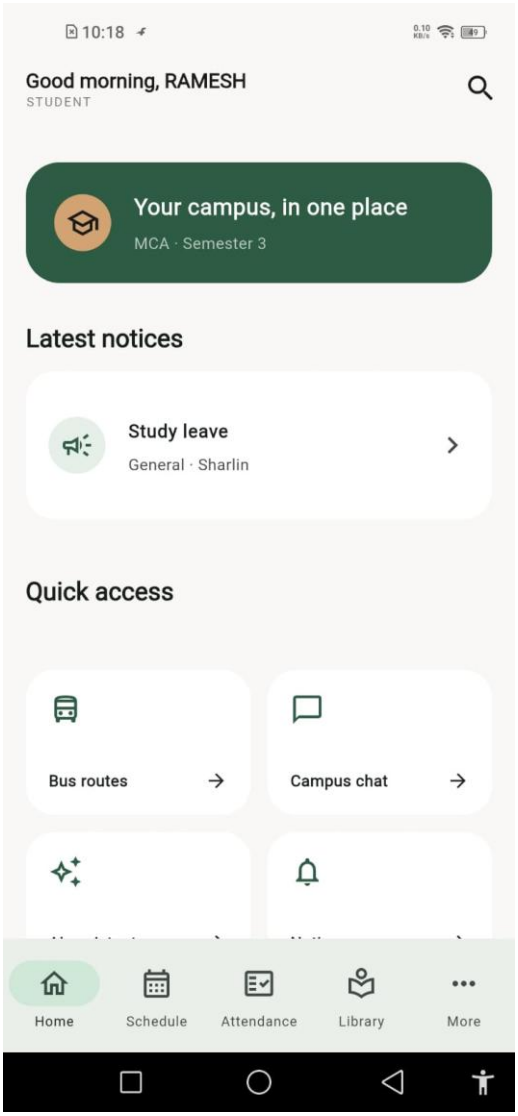


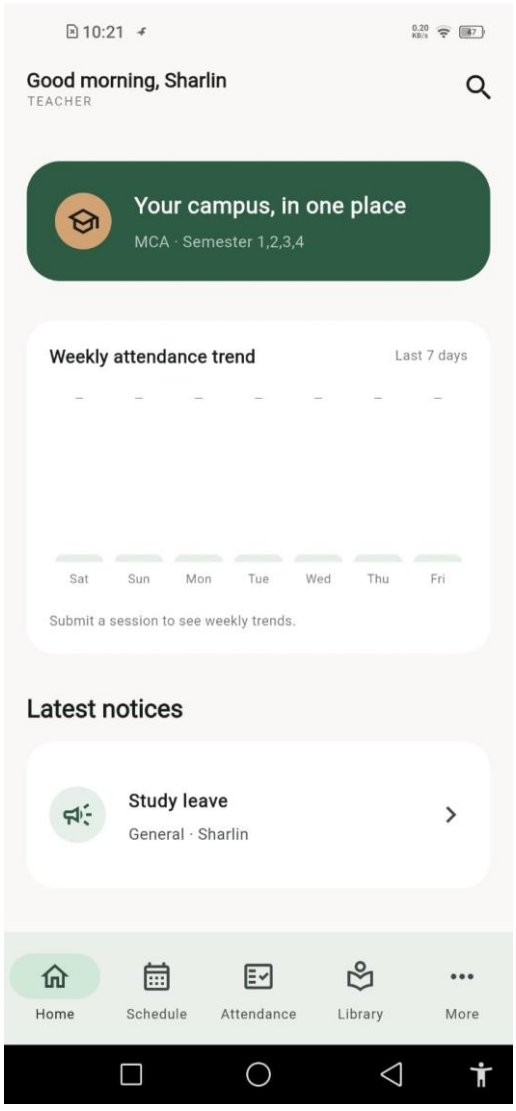
5.3 User Profile and Role Resolution

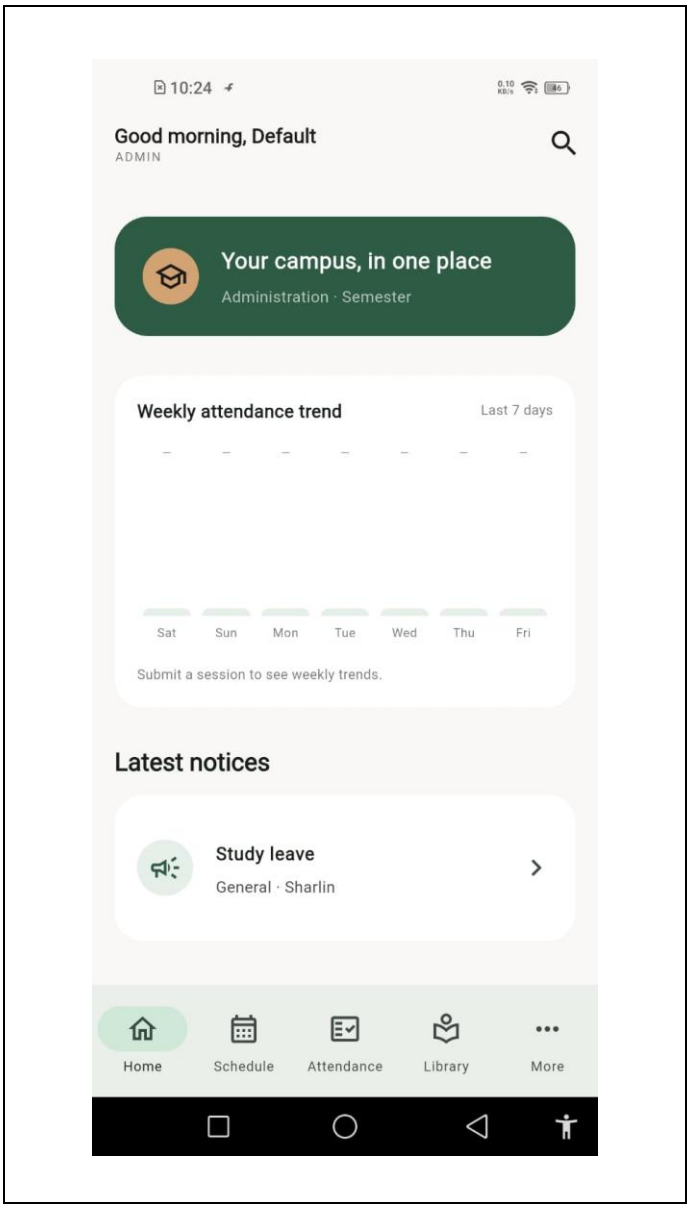
After sign-in, the app resolves profile information and role so that the navigation layer can present relevant destinations. The profile lookup must be consistent with Firestore security rules and the user records provisioned in the Firebase project.

5.4 Role-Based Home and Navigation

The application is intended for student, teacher, and administrator roles. Each role should see the screens and actions appropriate to its permissions. Backend authorisation must be maintained even when an action is hidden from the UI.







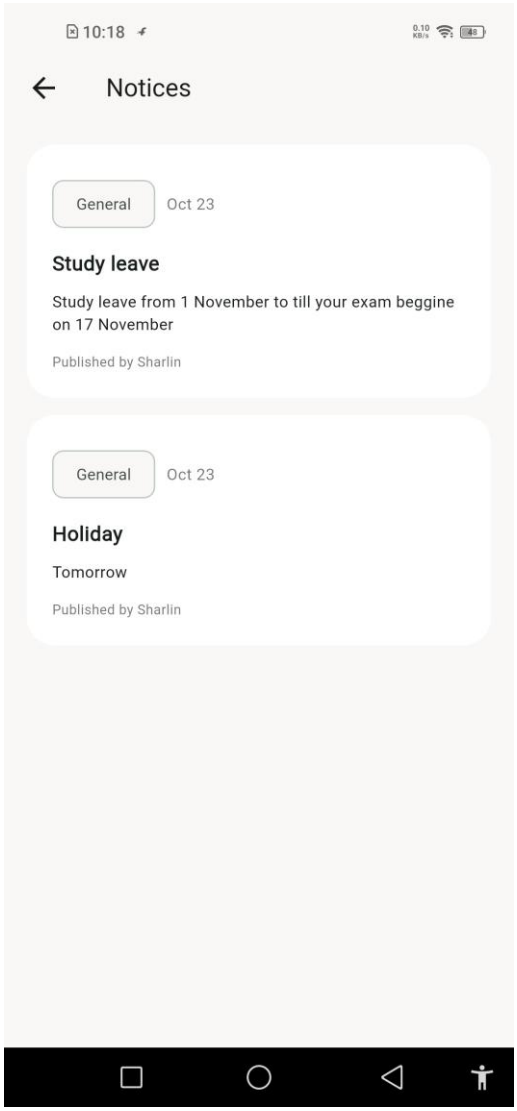
5.5 Notices Module

The notices module supports the display and staff management of campus notices. In the current tested behaviour, administrators can delete published notices and drafts, while teachers can delete their own published notices and drafts. Teachers cannot delete another teacher's notice, and students do not receive the delete option. Destructive actions should use a confirmation dialog and display feedback when the operation succeeds or fails.

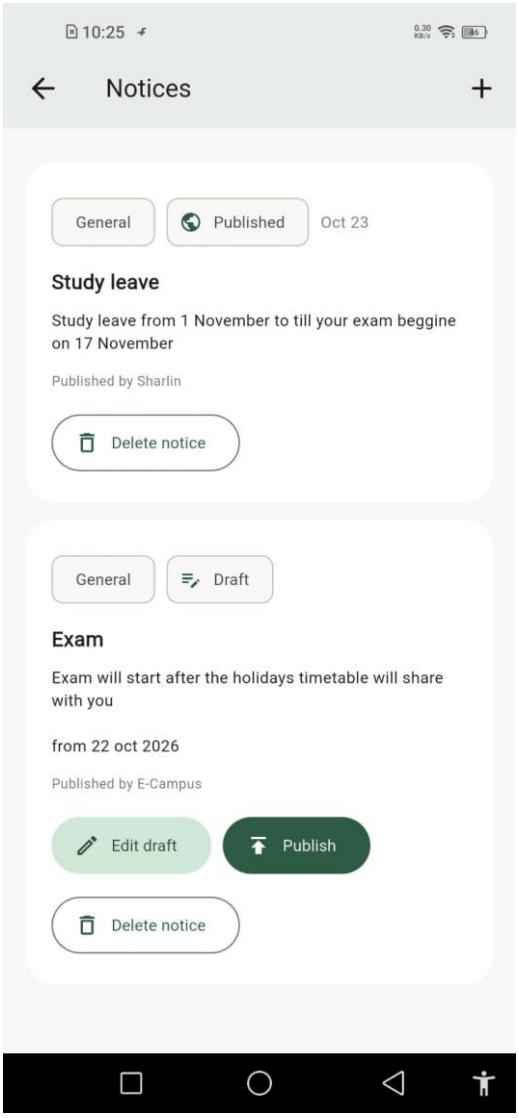
5.5.1 Notice Lifecycle

1. An authorised staff user creates a notice, if creation is enabled in the current build.
2. The notice is stored in Firestore with author and publication metadata.
3. A draft may be edited or published according to the current UI and rules.
4. Readers see notices permitted by their role and publication state.
5. A teacher may delete their own notice; an administrator may delete any notice.

6. Firestore rules enforce the final permission decision independently of the UI.



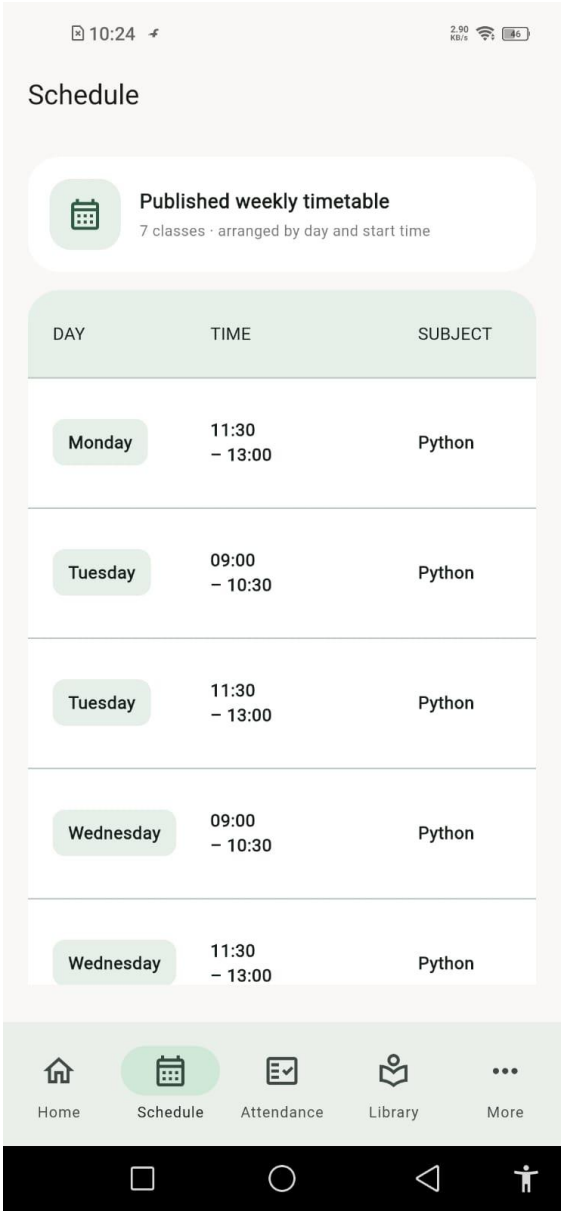
Student Notice



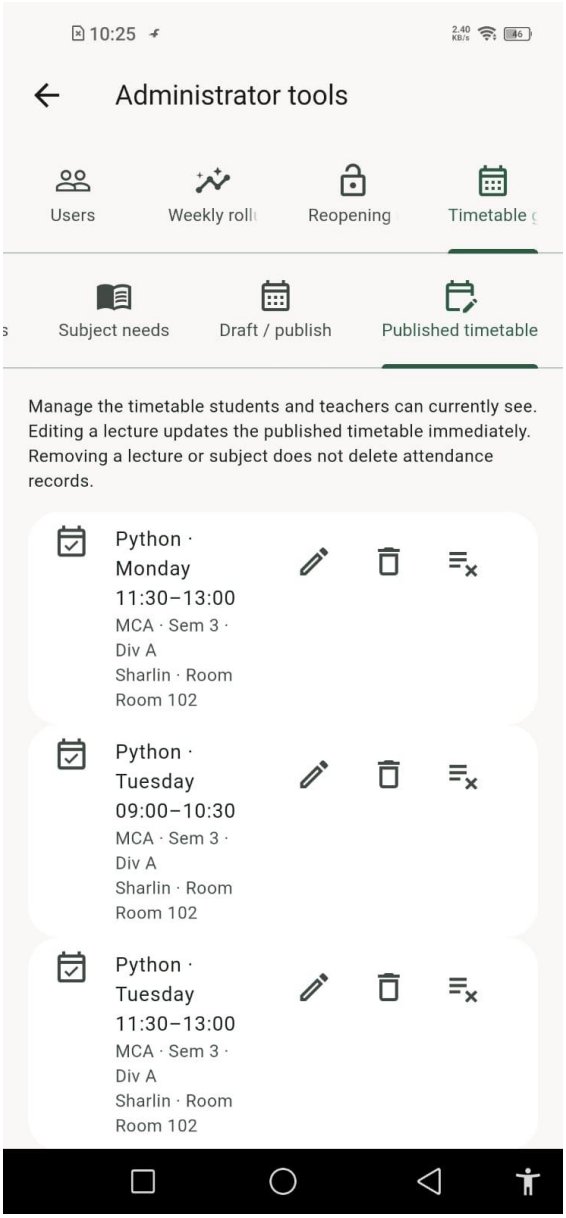
Admin Notice

5.6 Timetable Module

Timetable functionality has been part of the project scope and timetable-related data structures were referenced in prior development context. The final report should describe only the screens and actions that can be demonstrated. If timetable generation or publishing is incomplete, label it as in progress or future work rather than complete.



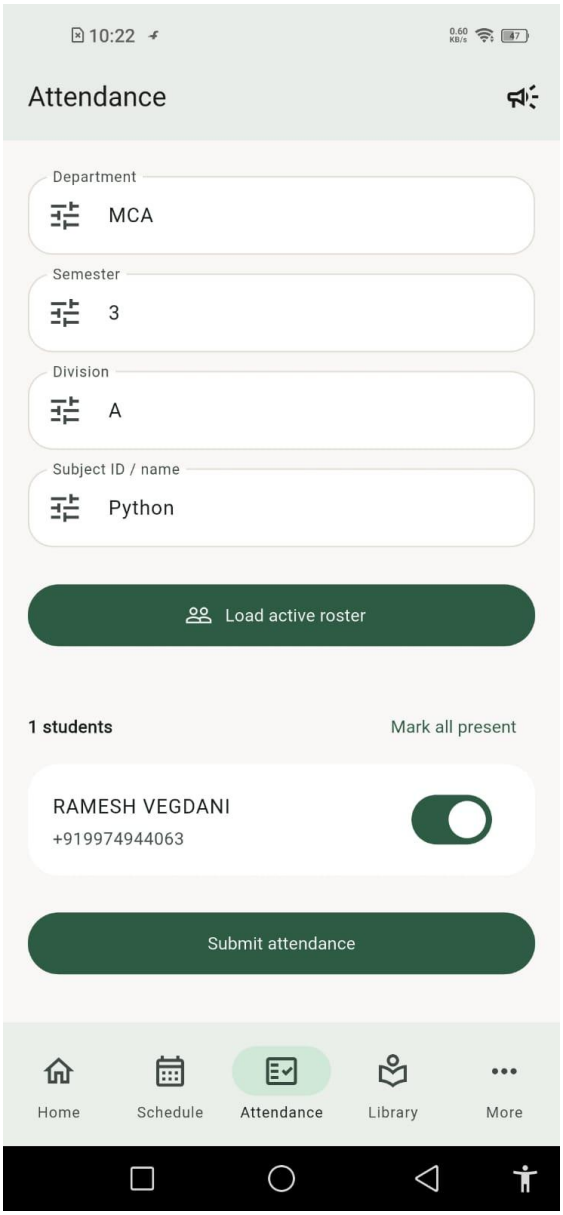
Student



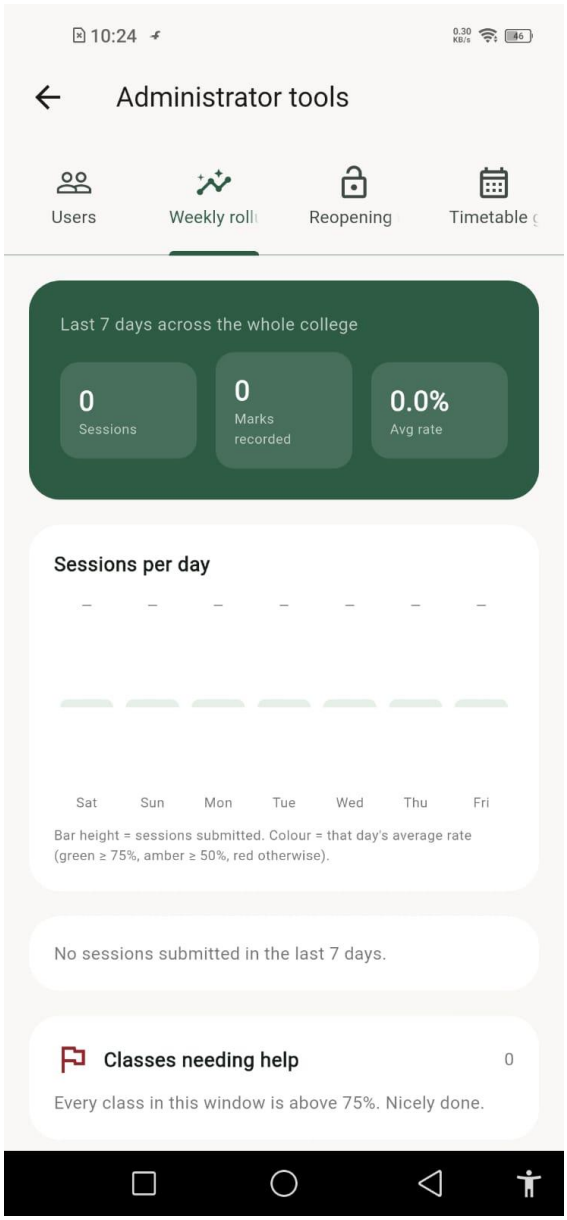
Admin

5.7 Attendance Module

Attendance screens have been referenced in the project context. The final implementation should be checked for how attendance records are entered, stored, aggregated, and presented. Do not state that calculations are accurate or complete until they have been tested with known sample records.



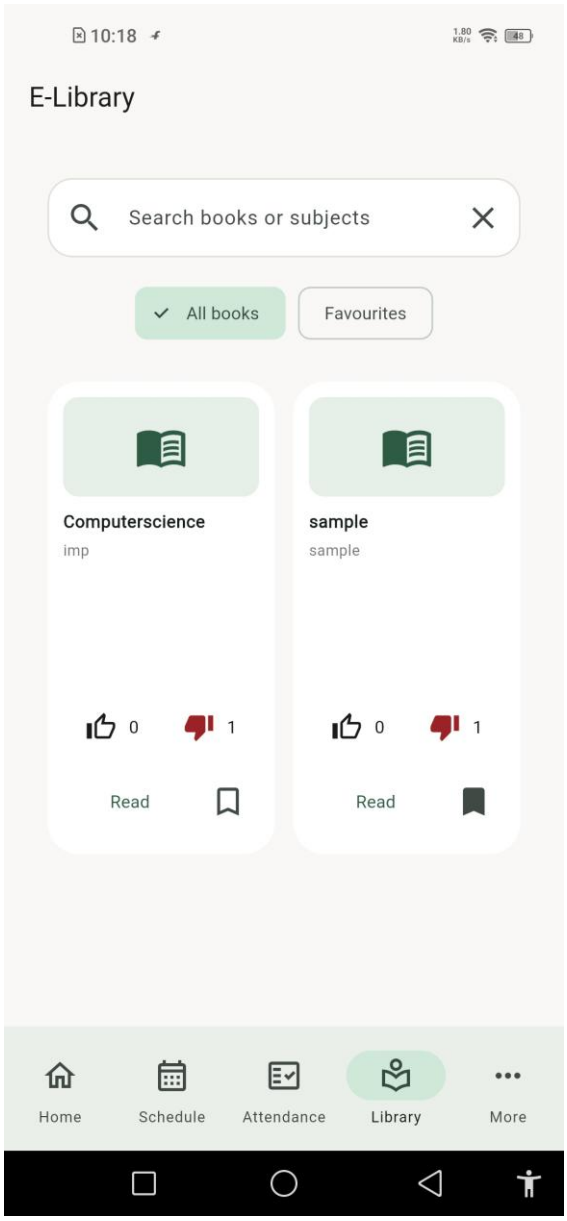
Teacher



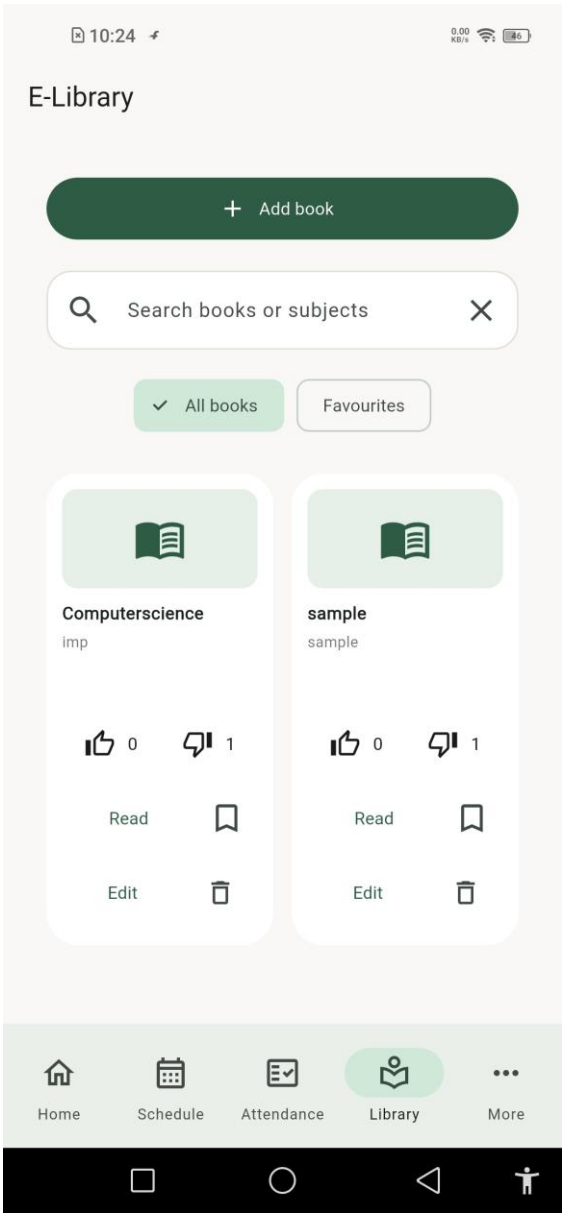
Admin

5.8 Library / E-Library Module

A library module has been referenced in the project scope. Document the actual functions present, such as browsing records, opening resources, or managing content. Verify the data source and any file-storage integration before including technical claims.



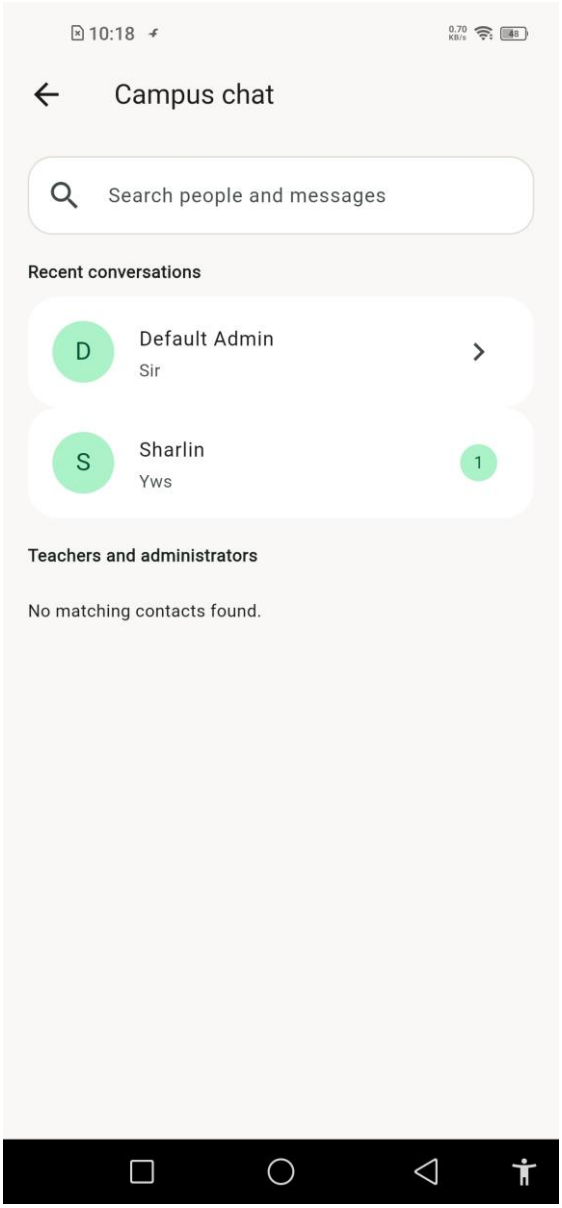
Student



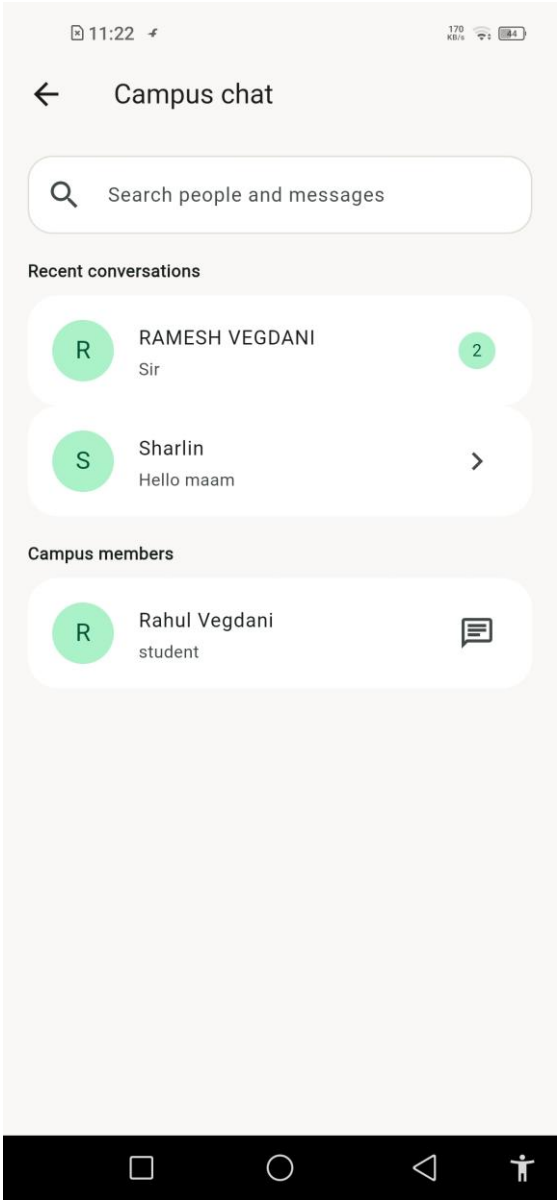
Admin

5.9 Campus Chat

Campus Chat was part of the planned project scope and earlier development included debugging contact and conversation permissions. Its current operational status must be confirmed before it is presented as completed. If the module is still incomplete, retain it in future enhancements or limitations.



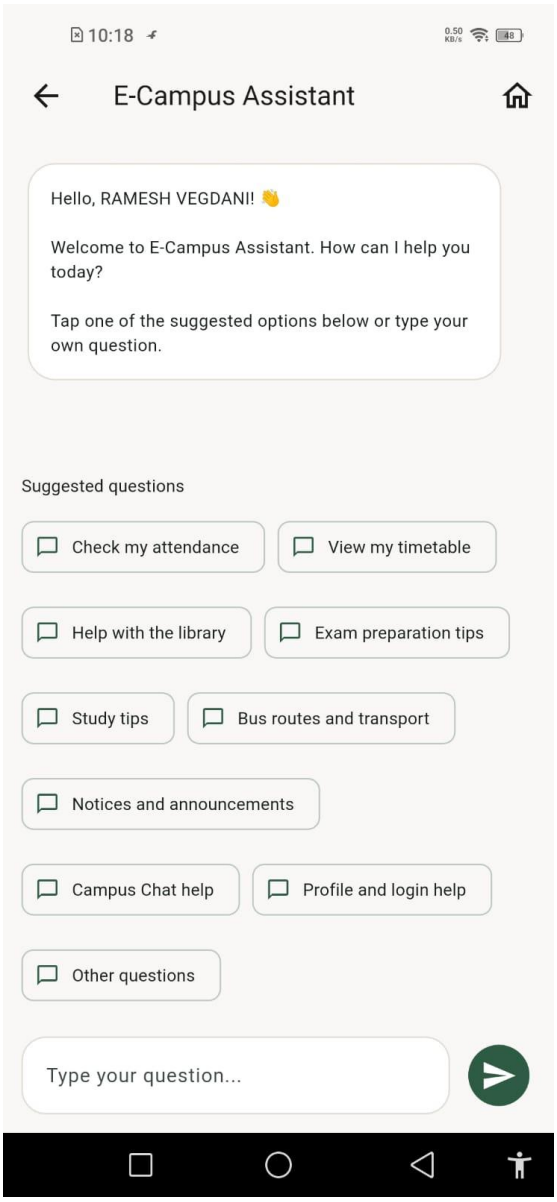
Student



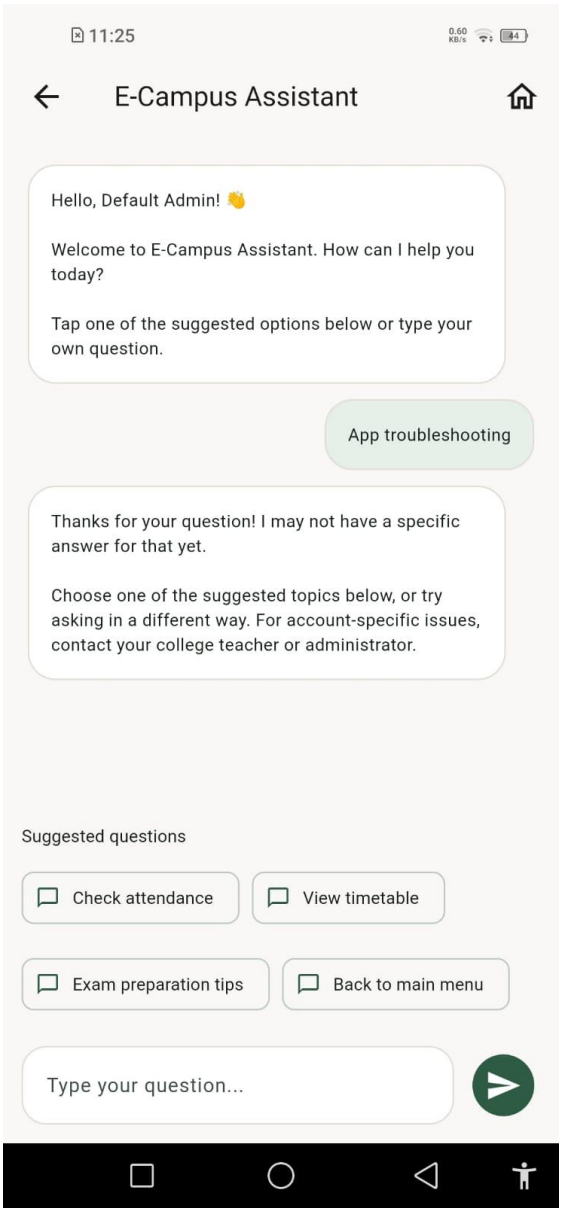
Admin

5.10 Notifications and AI Assistant

Firebase Messaging and an AI-related service are present in the known project dependency/context. Their end-to-end functionality must be verified. Do not claim push delivery, AI answer quality, or production availability without tests and the corresponding configuration.



Student



Admin

5.11 Project Source Structure

Path	Purpose based on known project context
lib/main.dart	Application entry point.
lib/app.dart	Application-level composition and shell; verify exact responsibilities.
lib/firebase_options.dart	Generated Firebase configuration; do not publish sensitive credentials.
lib/core/models/models.dart	Shared Dart models.
lib/core/services/data_repository.dart	Data access and repository operations.

lib/core/services/ai_service.dart	AI-related service integration, if enabled.
lib/core/services/notification_service.dart	Notification-related service integration, if enabled.
lib/core/theme/app_theme.dart	Shared app theme.
lib/features/auth/auth_screen.dart	Authentication screen.
lib/features/home/home_screen.dart	Home/dashboard screen.
lib/features/screens/screens.dart	Grouped application module screens, including notices.

5.12 Chapter Summary

This chapter describes the main modules and distinguishes confirmed notice behaviour from other modules that require final implementation verification.

CHAPTER 6: TESTING AND RESULTS

6.1 Testing Objectives

- Check that the app starts and initialises configured services.
- Verify the configured authentication workflow.
- Verify role-specific navigation.
- Confirm allowed and denied notice operations.
- Check data-loading screens and error handling.
- Record results from the exact build used for submission.

6.2 Testing Types

Test type	Purpose
Functional testing	Verify that screens and user actions behave as expected.
Role/permission testing	Check actions allowed and denied for each role.
Integration testing	Check the app's interaction with Firebase and any enabled external services.
Usability testing	Check navigation, labels, dialogs, and feedback.
Regression testing	Repeat core tests after code or rules changes.
Build testing	Verify debug/release build and installation on a supported device.

6.3 Test Cases

ID	Test case	Expected result	Current evidence / result
TC-01	Launch application with valid configuration	Application starts successfully	Pending: record on final build
TC-02	Sign in with configured method	Authenticated session is established	Pending: verify method and result
TC-03	Open as student	Student sees permitted screens and data	Pending
TC-04	Open as teacher	Teacher sees permitted staff screens	Pending
TC-05	Open as administrator	Administrator sees permitted admin screens	Pending
TC-06	Teacher deletes own draft notice	Notice is deleted; list updates	User reported notice deletion works
TC-07	Teacher deletes own published notice	Notice is deleted; list updates	User reported notice deletion works

TC-08	Teacher attempts to delete another teacher's notice	Action is unavailable or denied by rules	Confirm backend denial in final test
TC-09	Administrator deletes a draft notice	Notice is deleted; list updates	User reported notice deletion works
TC-10	Administrator deletes a published notice	Notice is deleted; list updates	User reported notice deletion works
TC-11	Student attempts notice deletion	No delete UI; backend denies operation	Verify backend rule test
TC-12	Load timetable	Permitted timetable data appears	Pending
TC-13	Load attendance	Permitted attendance data appears	Pending
TC-14	Load library resources	Permitted library data appears	Pending
TC-15	Force permission/network error	Clear error shown; no false success	Pending
TC-16	Run flutter analyze	No unresolved analysis issues	Run on final checkout

6.4 Test Environment Record

Item	**Value to complete**
App version / Git commit	[Enter version or commit]
Flutter version	[Run flutter --version]
Dart version	[Run dart --version]
Device / emulator	[Enter device model]
Android version	[Enter version]
Firebase project ID	[Confirm intended project]
Test date	[Enter date]
Tester(s)	[Enter group member(s)]

6.5 Results and Evidence

Only the notice deletion workflow has been reported as working perfectly in the latest conversation. Other results are intentionally marked pending because no final evidence for them was provided in the current report-building turn. Replace pending values with actual test observations and attach genuine screenshots.

6.6 Chapter Summary

The test plan defines the evidence required to demonstrate the system. A final submission should contain completed results for all modules included in the demonstration scope.

CHAPTER 7: INSTALLATION, CONFIGURATION AND EXECUTION

7.1 Prerequisites

- Flutter SDK installed and available on PATH.
- Android Studio and Android SDK tools installed.
- A compatible JDK installed for the project's Gradle configuration.
- A physical Android device with debugging enabled or a configured emulator.
- Access to the correct Firebase project and its configuration.
- The final E-Campus source folder and a valid pubspec.yaml.

7.2 Verify the Development Environment

Run these commands in a terminal opened at the project folder:

flutter doctor

flutter --version

dart --version

flutter devices

7.3 Firebase Configuration

The project context previously identified a Firebase project ID of e-campus-f025f and a generated lib/firebase_options.dart file. Confirm that these match the current submission. The app may have a platform-specific Firebase configuration; verify the Android application ID and the Firebase app registration. Do not include service-account keys or secret values in the report.

7. Confirm the intended Firebase project in Firebase Console.
8. Check lib/firebase_options.dart and platform-specific configuration.
9. Confirm the Android application ID matches the registered Firebase Android app.
10. Verify that Authentication providers and Firestore are enabled as required.
11. Publish the correct Firestore rules and verify role/ownership behaviour.

7.4 Install Dependencies

flutter pub get

flutter analyze

7.5 Run the Application

flutter devices

flutter run -d <device_id>

Use the device ID shown by flutter devices. If only one device is available, flutter run may select it automatically.

7.6 Build an Android APK

flutter build apk --release

The typical output path is build/app/outputs/flutter-apk/app-release.apk. Verify that the build completed and that the file exists. Release signing and distribution settings must be checked before sharing the APK.

7.7 Common Troubleshooting

Problem	Recommended checks
Firebase initialisation error	Check Firebase options, project ID, app ID, and platform configuration.
Firestore permission-denied	Check current user role, ownership fields, query constraints, and deployed rules in the correct project.
Empty list / missing contacts	Check Firestore data, query filters, account role, network, and permissions.
Gradle / JDK error	Check compatibility between Flutter, Gradle, Android Gradle Plugin, and installed JDK.
No device found	Run flutter devices, enable debugging, reconnect the device, or start an emulator.
Build succeeds but feature fails	Test runtime configuration and service permissions separately from build success.

7.8 Chapter Summary

This chapter gives a repeatable setup, execution, and build process. Commands should be run from the root of the final project checkout.

CHAPTER 8: LIMITATIONS AND FUTURE ENHANCEMENTS

8.1 Current Limitations

- The report is based on the project context and a reference report; the full current source repository has not been independently inspected in this drafting step.
- Exact Firestore collection schemas and indexes must be verified against the latest source and deployed rules.
- Campus Chat, push notifications, AI assistance, bus tracking, placement, grievance, canteen, and analytics must not be claimed as complete unless the final build demonstrates them.
- Test results other than notice deletion are pending verification in this report.
- External service behaviour depends on credentials, quotas, network connectivity, and provider availability.

8.2 Future Enhancements

- Complete and validate remaining in-progress campus modules.
- Improve automated tests for role permissions and Firestore security rules.
- Add robust monitoring and error reporting.
- Expand timetable and attendance reporting after verifying data integrity.
- Enhance notification workflows and delivery diagnostics.
- Extend AI assistant capabilities with privacy safeguards and source-grounded responses.
- Add bus route and location features only with valid mapping configuration and accurate tracking data.
- Improve accessibility, localisation, and usability based on student and staff feedback.

8.3 Conclusion

E-Campus provides a Flutter-based foundation for selected college management workflows. The project combines a mobile interface with Firebase services and role-aware access patterns. The notice deletion workflow has been confirmed by the group. Final academic submission should be supported by a build-specific source review, real screenshots, completed tests, and confirmation that each described feature is available in the submitted version.

REFERENCES

- Flutter Documentation. https://docs.flutter.dev/
- Dart Documentation. https://dart.dev/guides
- Firebase Documentation. https://firebase.google.com/docs
- Firebase Authentication Documentation. https://firebase.google.com/docs/auth
- Cloud Firestore Documentation. https://firebase.google.com/docs/firestore
- Riverpod Documentation. https://riverpod.dev/
- Android Developers Documentation. https://developer.android.com/
- GitHub Project Link: https://github.com/rahulvegdani/E-Campus-Flutter-Firebase